

HippoCortex

Biologically-Inspired Continual Learning via Sharp Wave Ripple Generative Replay in State Space Models

Hasinthaka Piyumal (SE/2021/036)
Induwara Mihisara (SE/2021/025)

Praveen Dedigama (SE/2021/031)
Thagya Kavindi (SE/2021/062)

Group 07 · Supervisor: Dr. Nalin Warnajith

1. Introduction

1.1 Background

We want machines that can keep learning new things over time, the way people do. The problem is that today’s neural networks forget. When you train them on a new task, they overwrite what they learned for the old one and their performance on it collapses (De Lange et al., 2022). The field calls this *catastrophic forgetting* (Kirkpatrick et al., 2017). Several workarounds exist, but none of them really fits a robot that has to keep learning for months on a small on-board computer.

1.2 Importance

A robot that ships with a frozen model cannot pick up a new room, a new object, or a new owner once it leaves the factory. Sending its data to a cloud server for re-training drains battery, leaks private camera footage, and does not work offline. If the robot can learn locally and its memory does not balloon as it sees more, we open the door to genuine lifelong adaptation for the next generation of personal and service robots.

1.3 Motivation

The mammalian brain has already solved this problem. While the animal is awake, the *hippocampus* (a fast memory area) quickly stores new experience. Later, during sleep, it sends short *compressed summaries* of those experiences to the *neocortex* (a slower, long-term store) for safekeeping (Buzsáki, 2015; McClelland et al., 1995). No raw recording is ever passed around. HippoCortex turns that idea into a system. We use a fast modern sequence model called Mamba (Gu & Dao, 2023) as the brain-like backbone, and instead of saving past data we keep tiny per-task summaries and let a small helper network recreate the missing experience whenever we need it.

2. Problem Statement

Existing approaches to continual learning fall into three groups, and each one is a poor fit for a deployed robot. The **rehearsal** group keeps a buffer of past data, or trains a generator that recreates that data, and replays it (Rebuffi et al., 2017; Shin et al., 2017); storage and privacy costs grow as the robot sees more. The **regularisation** group adds a penalty that tries to stop important weights from changing (Kirkpatrick et al., 2017); the penalty is fragile to tune—too strong and the model stops learning, too weak and it forgets anyway. The **projection** group, including the recent state-space methods Mamba-CL (Cheng et al., 2024) and Inf-SSM (Lee et al., 2025), asks the model to make all of its new learning in a "safe" direction that does not disturb older skills; this is light on memory but accuracy drops once the task stream gets long.

No published method delivers all three of the following at once: (i) the model never stores raw past inputs; (ii) the cost per training step is small enough that the method runs on a robot’s on-board chip; and (iii) accuracy stays competitive even after dozens of tasks. HippoCortex targets exactly this gap, by combining a tiny per-task summary, a small helper that recreates past experience from that summary, and a built-in safeguard that protects old skills during every weight update.

2.1 Research questions

The work is organised around four research questions. **RQ1:** can a tiny per-task summary (a few statistics, kilobytes per task) really replace a buffer of past inputs without hurting accuracy on long task streams? **RQ2:** can a small helper network recreate past experience from that summary

closely enough that a classifier trained on the recreated experience transfers cleanly to real past inputs? **RQ3:** does combining the two ideas above—compressed replay and a built-in safeguard against overwriting—do better than either idea on its own across twenty or more sequential tasks? **RQ4:** does the resulting model fit the speed and memory budget of the Go2 Edu’s on-board chip while running at well under 50 ms per frame under a real-world task schedule?

3. Objectives

- O1. Tiny per-task summary.** A small chunk of memory per task that captures the gist of what the network learned, without keeping any raw input. Target: the size of the summary stays the same regardless of how many training samples the task has.
- O2. Memory replay helper.** A small helper network that, given a task name, recreates synthetic versions of the experience the model had during that task. Target: a classifier trained on the recreated experience reaches within two percentage points of one trained on the real past data.
- O3. Built-in safeguard.** A simple step in training that catches changes which would damage what older tasks rely on and removes them before they reach the weights. Target: less than three percentage-point drop in old-task accuracy after twenty sequential tasks on Split-CIFAR100.
- O4. Validation on standard benchmarks.** Evaluate on two well-known continual-learning benchmarks (Split-CIFAR100 and ImageNet-R). Target: 8–10% higher average accuracy than the closest published baseline (Mamba-CL) and around 4% higher than the next one (Inf-SSM).
- O5. Robot deployment.** Add a Transformer-style block (Vaswani et al., 2017) to handle camera input and run the whole system on the Unitree Go2 Edu robot. Target: each frame processed in under 50 ms on the robot’s on-board chip.
- O6. Publication.** Two peer-reviewed papers: one on the Stage 1 algorithm, one on the Stage 2 robot deployment.

4. Preliminary Literature Review

4.1 Catastrophic forgetting

Kirkpatrick et al. (2017) introduced the most-cited modern fix, Elastic Weight Consolidation. The idea is to ask the network to keep its weights close to where they ended up after the previous task, with extra resistance to changing the weights that mattered most for that task. It works on short task streams but starts to misbehave as tasks pile up (De Lange et al., 2022).

4.2 Rehearsal and generative replay

iCaRL (Rebuffi et al., 2017) simply keeps a small buffer of past examples and rehearses them along with the new task; storage grows with the number of classes seen. Deep Generative Replay (Shin et al., 2017) replaces the buffer with a generator that recreates past inputs, but training a high-quality image generator on the fly on an edge device is not realistic. HippoCortex follows the same generative-replay idea but recreates the model’s *internal experience* instead of the raw image, which keeps the helper network small enough to run on the robot.

4.3 Methods that protect old skills directly

A second line of work asks the model to make every weight update in a direction that simply cannot disturb what older tasks relied on (Saha et al., 2021). The recent state-space variants Mamba-CL (Cheng et al., 2024) and Inf-SSM (Lee et al., 2025) apply the same idea to fast modern sequence models. They keep no buffer at all, but their accuracy still slips on long task streams. HippoCortex adds the recreated-experience signal on top of this safeguard, so the two mechanisms cover each other’s weaknesses.

4.4 Modern sequence models

Mamba (Gu & Dao, 2023) is a recent sequence model that matches Transformer-quality accuracy while running in linear time and using a small, constant amount of memory per step. That is exactly the profile we need on a robot: long memory of past observations without the heavy cost

that attention-based models pay.

4.5 Biological grounding

Buzsáki (2015) reviews twenty years of evidence that the hippocampus replays its memories during sleep, and McClelland et al. (1995) explains why a fast memory and a slow memory together avoid interference: they exchange short summaries, not the raw experience itself. HippoCortex copies the same division of labour (Figure 1).

4.6 Research gap

Across the strands above, no published method offers all of: no raw data stored, fast on-device training, and stable accuracy after many sequential tasks, all validated on a real robot. Figure 2 lays out the gap HippoCortex fills.

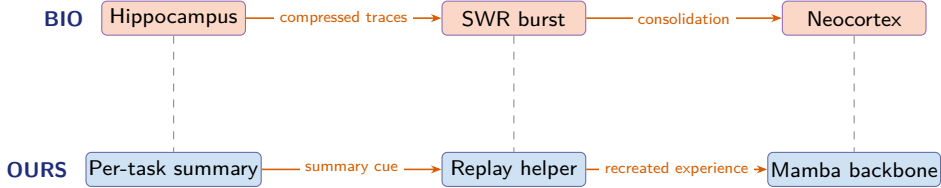


Figure 1: Mapping of biological consolidation onto HippoCortex components.

Method	Replay style	Raw data?	Memory growth	On robot?
EWC (Kirkpatrick et al., 2017)	none	no	small but task-aware	no
iCaRL (Rebuffi et al., 2017)	real past examples	yes	grows with classes	no
DGR (Shin et al., 2017)	recreates raw pixels	no	generator-sized	no
GPM (Saha et al., 2021)	none, safeguard only	no	grows with tasks	no
Mamba-CL (Cheng et al., 2024)	none, safeguard only	no	grows with tasks	no
Inf-SSM (Lee et al., 2025)	none, safeguard only	no	grows with tasks	no
HippoCortex (ours)	internal experience + safeguard	no	few KB per task	yes (Go2)

Figure 2: Comparison of continual-learning methods. HippoCortex is the only entry that combines latent generative replay with null-space protection on a linear-time SSM backbone, with planned validation on a physical robot.

5. Methodology

The project runs in two stages. **Stage 1** (Months 1–3) builds the core algorithm and validates it on standard image benchmarks. **Stage 2** (Months 4–10) extends the model to a hybrid architecture and deploys it on the Unitree Go2 Edu quadruped. Hardware access for Stage 2 is already in hand: the team’s separate hardware proposal to the WSO2 Call for Proposals on Unitree Go2 Edu Robot-based Projects has been *approved*, which gives us access to the robot at the WSO2 Colombo office along with a project grant.

5.1 Stage 1: Core system (Months 1–3)

HippoCortex chains four pieces, shown in Figure 3:

- **Mamba backbone.** A fast modern sequence network (Gu & Dao, 2023) that does the actual learning.
- **Per-task summary.** A tiny record we keep about each task we have seen, capturing only what the model’s internal features looked like for that task. We never keep raw images.
- **Replay helper.** A small generator network (Sohn et al., 2015) that, given a task name, recreates how the backbone’s internals would have looked during that task.
- **Built-in safeguard.** A simple step that we apply on every weight update. It catches the part of the update that would hurt older skills and removes it before the update is applied (Cheng et al., 2024; Saha et al., 2021).

Training loop (per new task). For each new task we run three short passes. (i) A *warm-up* on the new task alone, so the backbone gets a sensible starting point. (ii) A *joint pass* mixes the new

task with recreated past experience from the replay helper, and routes every weight update through the safeguard. (iii) A *consolidation* pass updates the per-task summary with the new task and tells the safeguard which directions to protect from now on.

Benchmarks, baselines, metrics.

Datasets	Split-CIFAR100 (twenty 5-class tasks) and ImageNet-R (a harder distribution shift).
Baselines	Naive fine-tuning, EWC (Kirkpatrick et al., 2017), DGR (Shin et al., 2017), Mamba-CL (Cheng et al., 2024), Inf-SSM (Lee et al., 2025).
Metrics	Average accuracy across tasks, forgetting rate, and memory used (in MB) as the number of tasks grows.

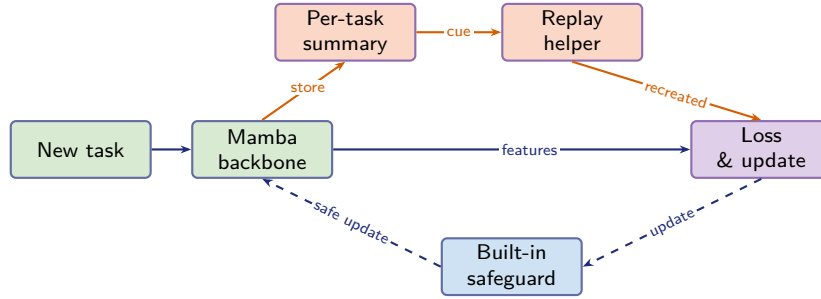


Figure 3: The Stage 1 HippoCortex system. Recreated past experience from the replay helper is mixed with the current task; every weight update passes through the safeguard before reaching the backbone, so older skills stay intact.

5.2 Stage 2: Robot deployment (Months 4–6)

The Mamba backbone alone does not make full use of spatial structure in camera frames, so for the robot we add a Transformer-style block (Vaswani et al., 2017) between two Mamba layers. The Mamba layers handle the time dimension (the stream of frames), and the attention block helps with the visual content of each frame. The same HippoCortex safeguard and replay helper wrap the whole thing, so we do not need to change anything else. The model runs on the Go2 Edu’s on-board chip (Jetson Orin NX 16 GB, 100 TOPS). On the robot we test it on four tasks back-to-back: recognising terrain, interacting with an object, staying aware of nearby people, and mapping a new room. The implementation uses standard tools: Python 3.11, PyTorch 2.x, ROS2, and Mujoco / Isaac Sim for pre-deployment testing in simulation.

6. Expected Project Deliverables

- **Open-source reference implementation** (PyTorch, GitHub) of the full HippoCortex stack with reproducible training scripts and pre-trained checkpoints.
- **Stage 1 benchmark report** on Split-CIFAR100 and ImageNet-R: average accuracy, forgetting rate, memory footprint, and full per-component ablations against EWC, DGR, Mamba-CL and Inf-SSM.
- **Stage 1 paper** (peer-reviewed) describing the core algorithm and Stage 1 results.
- **Hybrid Mamba + Transformer extension** that wraps the SWR module around the hybrid stack for richer visual input.
- **Embodied demonstration** on the Go2 Edu running the four-task curriculum (terrain, object interaction, human proximity, environment mapping) on-device, with logged latency and per-task accuracy.
- **Stage 2 paper and thesis chapter** documenting the full embodied evaluation.

7. Project Schedule

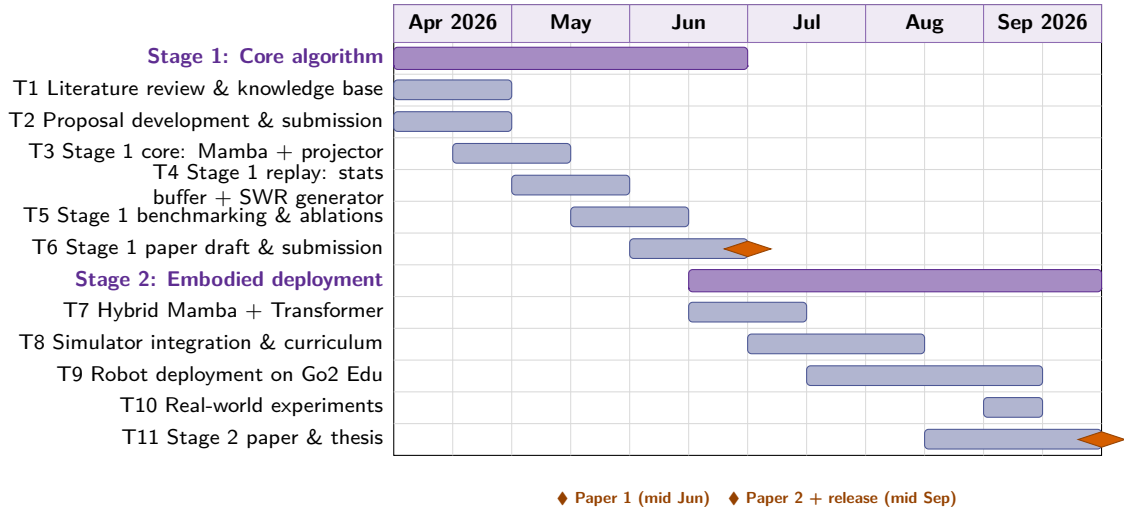


Figure 4: Six-month project schedule (April–September 2026). Each column is a half-month. Stage 1 delivers the core algorithm and first paper by late June; Stage 2 covers the hybrid extension, simulator validation, on-device robot deployment, real-world experiments, and the final paper / thesis.

7.1 Team responsibilities

The Gantt above lists each task’s lead. Per-member ownership is summarised below.

Member	ID	Primary responsibility
Hasinthaka Piyumal	SE/2021/036	Team lead, research direction, training pipeline, paper writing, WSO2 liaison.
Praveen Dedigama	SE/2021/031	Mamba backbone, null-space projector, hybrid stack design.
Induwara Mihisara	SE/2021/025	Biological grounding, data loaders, simulator integration.
Thagya Kavindi	SE/2021/062	Evaluation metrics, benchmark runs, ablation studies, results analysis.

8. References

- Buzsáki, G. (2015). Hippocampal sharp wave-ripple: A cognitive biomarker for episodic memory and planning. *Hippocampus*, 25(10), 1073–1188.
- Cheng, D., Huang, Y., Zhang, J., Chen, K., Liu, M., & Huang, Q. (2024). Mamba-CL: Optimizing selective state space model in null space for continual learning. *arXiv preprint arXiv:2411.15469*.
- De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., & Tuytelaars, T. (2022). A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7), 3366–3385.
- Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
- Lee, I. N., Mahmoodi, L., Le, T., & Harandi, M. (2025). Exemplar-free continual learning for state space models. *arXiv preprint arXiv:2505.18604*.
- McClelland, J. L., McNaughton, B. L., & O’Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3), 419–457.
- Rebuffi, S.-A., Kolesnikov, A., Sperl, G., & Lampert, C. H. (2017). iCaRL: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2001–2010).
- Saha, G., Garg, I., & Roy, K. (2021). Gradient projection memory for continual learning. In *International Conference on Learning Representations (ICLR)*.
- Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Sohn, K., Lee, H., & Yan, X. (2015). Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.